```
import { useState, useEffect, useRef, useCallback } from "react";

// ── ROOMS ──────────────────────────────────────────────────────────────
const ROOMS = {
  command:   { id:"command",   label:"COMMAND CENTER",       short:"CMD", color:"
  stock:     { id:"stock",     label:"STOCK ROOM",           short:"STK", color:"
  orders:    { id:"orders",    label:"ORDER CONTROL",        short:"ORD", color:"
  packaging: { id:"packaging", label:"PACKAGING & SHIPPING", short:"PKG", color:"
  storage:   { id:"storage",   label:"STORAGE ROOM",         short:"STO", color:"
  design:    { id:"design",    label:"DESIGNERS & EDITORS",  short:"DES", color:"
};

const CONVEYORS = [
  { id:"c1", from:"stock",    to:"packaging", color:"#00aaff", points:[[115,150],
  { id:"c2", from:"orders",   to:"packaging", color:"#ff6600", points:[[680,150],
  { id:"c3", from:"packaging",to:"storage",   color:"#cc44ff", points:[[210,390],
  { id:"c4", from:"stock",    to:"design",    color:"#44ffcc", points:[[200,90],[
];

const ROOM_ACTIONS = {
  command:   ["Monitoring systems","Reviewing dashboard","Dispatching orders","Pl
  stock:     ["Counting inventory","Restocking shelves","Scanning barcodes","Sort
  orders:    ["Processing order","Verifying details","Flagging rush order","Updat
  packaging: ["Packing item","Sealing box","Printing label","Loading belt","Dispa
  storage:   ["Sorting crates","Auditing stock","Moving pallets","Filing records"
  design:    ["Editing artwork","Exporting files","Creating mockup","Reviewing de
};

// ── INITIAL STOCK ──────────────────────────────────────────────────────
const INITIAL_STOCK = [
  { id:"s1", name:"Hoodies",   qty:2,  price:35.00, emoji:"🧥", category:"Clothin
  { id:"s2", name:"T-Shirts",  qty:3,  price:18.00, emoji:"👕", category:"Clothin
  { id:"s3", name:"Caps",      qty:5,  price:12.00, emoji:"🧢", category:"Accessc
];

// ── WORKERS ────────────────────────────────────────────────────────────
const WORKER_DEFS = [
  { id:0,  name:"Dee Boss",  role:"Supreme Commander", roomId:"command",   color:
  { id:1,  name:"Marcus",    role:"Stock Manager",     roomId:"stock",     color:
  { id:2,  name:"Priya",     role:"Inventory Clerk",   roomId:"stock",     color:
  { id:3,  name:"Cruz",      role:"Order Processor",   roomId:"orders",    color:
  { id:4,  name:"Michaella", role:"Queue Manager",     roomId:"orders",    color:
  { id:5,  name:"Leila",     role:"Pack Specialist",   roomId:"packaging", color:
  { id:6,  name:"Diego",     role:"Shipping Agent",    roomId:"packaging", color:
```

```
  { id:7,  name:"Yuki",      role:"Label Printer",     roomId:"packaging", color:
  { id:8,  name:"Tobias",    role:"Storage Clerk",     roomId:"storage",   color:
  { id:9,  name:"Amara",     role:"Stock Auditor",     roomId:"storage",   color:
  { id:10, name:"Ren",       role:"Lead Designer",     roomId:"design",    color:
  { id:11, name:"Cassidy",   role:"Content Editor",    roomId:"design",    color:
];

// ── ANIMATED ACTIONS per role ──────────────────────────────────────────────
const ANIM_ACTIONS = {
  command:   ["💻 Typing report","📊 Reviewing charts","📞 On call","✅ Approvin
  stock:     ["📦 Unloading truck","🔍 Scanning item","📋 Counting stock","🚚 Mo
  orders:    ["⌨️ Typing order","📞 Calling customer","🖨️ Printing slip","✅ Con
  packaging: ["📦 Placing on belt","🎁 Wrapping item","🏷️ Printing label","📪 Se
  storage:   ["📦 Shelving box","🔢 Counting items","📋 Logging stock","🏗️ Movin
  design:    ["🖥️ Editing design","✏️ Sketching","🗂️ Exporting file","🎨 Colour
};

function rand(a,b){return Math.floor(Math.random()*(b-a+1))+a;}
function randEl(arr){return arr[rand(0,arr.length-1)];}
function fmt(s){return new Date(s*1000).toISOString().substr(11,8);}

function initWorker(def){
  const room=ROOMS[def.roomId];
  return {
    ...def, action:randEl(ROOM_ACTIONS[def.roomId]),
    animAction:randEl(ANIM_ACTIONS[def.roomId]),
    progress:rand(0,80), status:def.isBoss?"active":randEl(["active","active","bu
    efficiency:def.isBoss?100:rand(72,99), tasksToday:rand(0,20),
    wx:room.x+rand(28,room.w-48), wy:room.y+rand(28,room.h-38),
    tx:0,ty:0, walking:false, walkPhase:rand(0,100),
    paused:false, mood:randEl(["😊","😐","💪","🤔","😄"]),
    animFrame:0,
  };
}

function mkBox(cid){return{id:Math.random(),conveyorId:cid,t:rand(0,60)/100,color

// ── CHARACTER ──────────────────────────────────────────────────────────────
function Character({w,onClick,isSelected}){
  const bob=Math.sin(w.walkPhase*0.15)*(w.walking?2.5:0.4);
  const sc=w.isBoss?1.3:1;
  const col=w.color;
  const sCol={active:"#00ff88",busy:"#ffaa00",idle:"#444",paused:"#ff4444"}[w.sta
  return(
    <g transform={`translate(${w.wx},${w.wy})`} onClick={()=>onClick(w.id)} style
      {isSelected&&<circle cx={0} cy={-14*sc} r={20*sc} fill="none" stroke={col}
      <g transform={`translate(0,${bob})`}>
```

```
        <ellipse cx={0} cy={2} rx={8*sc} ry={2.5*sc} fill="#000" opacity={0.25}/>
        {/* legs */}
        <rect x={-5*sc} y={0} width={4*sc} height={10*sc} rx={2} fill={col} opaci
          transform={w.walking?`rotate(${Math.sin(w.walkPhase*0.28)*18},${-3*sc},
        <rect x={1*sc}  y={0} width={4*sc} height={10*sc} rx={2} fill={col} opaci
          transform={w.walking?`rotate(${-Math.sin(w.walkPhase*0.28)*18},${3*sc},
        {/* body */}
        <rect x={-8*sc} y={-20*sc} width={16*sc} height={20*sc} rx={3} fill={col}
        <rect x={-3*sc} y={-19*sc} width={6*sc} height={16*sc} rx={1} fill="#000"
        {/* arms */}
        <rect x={-13*sc} y={-18*sc} width={5*sc} height={10*sc} rx={2} fill={col}
          transform={w.walking?`rotate(${Math.sin(w.walkPhase*0.28)*22},${-10*sc}
        <rect x={8*sc}  y={-18*sc} width={5*sc} height={10*sc} rx={2} fill={col}
          transform={w.walking?`rotate(${-Math.sin(w.walkPhase*0.28)*22},${12*sc}
        {/* head */}
        <circle cx={0} cy={-27*sc} r={9*sc} fill="#FDBCB4"/>
        <ellipse cx={0} cy={-34*sc} rx={9*sc} ry={5*sc} fill={col} opacity={0.9}/
        <circle cx={-3*sc} cy={-28*sc} r={1.5*sc} fill="#333"/>
        <circle cx={3*sc}  cy={-28*sc} r={1.5*sc} fill="#333"/>
        <path d={`M${-3*sc} ${-24*sc} Q0 ${-22*sc} ${3*sc} ${-24*sc}`} stroke="#5
        {w.isBoss&&<text x={0} y={-41*sc} textAnchor="middle" fontSize={11}>👑</t
        <circle cx={8*sc} cy={-34*sc} r={3*sc} fill={sCol} stroke="#000" strokeWi
      </g>
      {/* speech bubble for anim action */}
      {isSelected&&(
        <g transform={`translate(12,${-40*sc})`}>
          <rect x={0} y={-12} width={80} height={15} rx={3} fill="#000" opacity={
          <text x={4} y={-1} fontSize={7} fill={col} fontFamily="'Courier New',mo
        </g>
      )}
      <rect x={-22} y={8} width={44} height={13} rx={2} fill="#000" opacity={0.75
      <text x={0} y={18} textAnchor="middle" fontSize={7.5} fill={col} fontFamily
    </g>
  );
}

// ── CONVEYOR ──────────────────────────────────────────────────────────────
function ConveyorBelt({conv,boxes,tick}){
  const pts=conv.points;
  const pathD=pts.map((p,i)=>i===0?`M${p[0]},${p[1]}`:`L${p[0]},${p[1]}`).join("
  let len=0;
  for(let i=1;i<pts.length;i++) len+=Math.sqrt((pts[i][0]-pts[i-1][0])**2+(pts[i]
  function ptAt(t){
    const target=t*len; let acc=0;
    for(let i=1;i<pts.length;i++){
      const sl=Math.sqrt((pts[i][0]-pts[i-1][0])**2+(pts[i][1]-pts[i-1][1])**2);
      if(acc+sl>=target){const f=(target-acc)/sl;return[pts[i-1][0]+(pts[i][0]-pt
```

```
      acc+=sl;
    }
    return pts[pts.length-1];
  }
  const off=(tick*3)%20;
  return(
    <g>
      <path d={pathD} stroke="#2a2a2a" strokeWidth={16} fill="none" strokeLinecap
      <path d={pathD} stroke="#1a1a1a" strokeWidth={14} fill="none" strokeLinecap
      <path d={pathD} stroke={conv.color} strokeWidth={2} fill="none" strokeLinec
      {boxes.filter(b=>b.conveyorId===conv.id).map(b=>{
        const[bx,by]=ptAt(Math.min(b.t,0.97));
        return(
          <g key={b.id} transform={`translate(${bx},${by})`}>
            <rect x={-7} y={-7} width={14} height={14} rx={1} fill={b.color} stro
            <line x1={-7} y1={0} x2={7} y2={0} stroke="#5C3317" strokeWidth={0.5}
            <line x1={0} y1={-7} x2={0} y2={7} stroke="#5C3317" strokeWidth={0.5}
          </g>
        );
      })}
    </g>
  );
}

// ── ROOM ──────────────────────────────────────────────────────────────────
function RoomBlock({room,onClick,isSelected,stockItems}){
  const col=room.color;
  return(
    <g onClick={()=>onClick(room.id)} style={{cursor:"pointer"}}>
      <rect x={room.x} y={room.y} width={room.w} height={room.h} fill={room.bg} s
      {/* floor tiles */}
      {Array.from({length:Math.ceil(room.w/25)}).map((_,i)=>Array.from({length:Ma
        <rect key={`${i}-${j}`} x={room.x+i*25} y={room.y+j*25} width={24} height
      )))}
      {/* header */}
      <rect x={room.x} y={room.y} width={room.w} height={20} fill={col} opacity={
      <text x={room.x+room.w/2} y={room.y+13} textAnchor="middle" fontSize={8.5}
      {/* corners */}
      {[[room.x,room.y,1,1],[room.x+room.w,room.y,-1,1],[room.x,room.y+room.h,1,-
        <g key={i}><line x1={cx} y1={cy} x2={cx+sx*9} y2={cy} stroke={col} stroke
      ))}
      {/* delivery truck in stock room */}
      {room.id==="stock"&&(
        <g transform={`translate(${room.x+room.w-55},${room.y+room.h-45})`}>
          <rect x={0} y={10} width={50} height={28} rx={2} fill="#334455" stroke=
          <rect x={32} y={4} width={18} height={18} rx={2} fill="#223344" stroke=
          <circle cx={10} cy={38} r={5} fill="#111" stroke="#555" strokeWidth={1}
```

```
          <circle cx={38} cy={38} r={5} fill="#111" stroke="#555" strokeWidth={1}
          <text x={16} y={28} fontSize={9}>🚚</text>
        </g>
      )}
      {/* computer desks in design/orders/command */}
      {(room.id==="design"||room.id==="orders"||room.id==="command")&&(
        Array.from({length:room.id==="command"?1:2}).map((_,i)=>(
          <g key={i} transform={`translate(${room.x+20+i*65},${room.y+room.h-35})
            <rect x={0} y={0} width={40} height={5} rx={1} fill="#223344"/>
            <rect x={5} y={-20} width={30} height={20} rx={2} fill="#112233" stro
            <rect x={8} y={-17} width={24} height={14} rx={1} fill="#001a2a" opac
            <text x={20} y={-7} textAnchor="middle" fontSize={7}>💻</text>
          </g>
        ))
      )}
      {/* storage room — Etsy stock boxes */}
      {room.id==="storage"&&stockItems&&(
        <g>
          {/* Box 1 label */}
          <rect x={room.x+10} y={room.y+28} width={70} height={50} rx={2} fill="#
          <rect x={room.x+10} y={room.y+28} width={70} height={14} rx={2} fill="#
          <text x={room.x+45} y={room.y+38} textAnchor="middle" fontSize={7} fill
          {stockItems.slice(0,2).map((s,i)=>(
            <text key={s.id} x={room.x+14} y={room.y+52+i*11} fontSize={7} fill="
          ))}
          {/* Box 2 label */}
          <rect x={room.x+90} y={room.y+28} width={70} height={50} rx={2} fill="#
          <rect x={room.x+90} y={room.y+28} width={70} height={14} rx={2} fill="#
          <text x={room.x+125} y={room.y+38} textAnchor="middle" fontSize={7} fil
          {stockItems.slice(2).map((s,i)=>(
            <text key={s.id} x={room.x+94} y={room.y+52+i*11} fontSize={7} fill="
          ))}
          {/* Pallet */}
          <rect x={room.x+10} y={room.y+82} width={150} height={8} rx={1} fill="#
          {[0,1,2,3,4].map(i=><line key={i} x1={room.x+20+i*28} y1={room.y+82} x2
        </g>
      )}
      {/* packaging table */}
      {room.id==="packaging"&&(
        <g>
          <rect x={room.x+15} y={room.y+room.h-45} width={room.w-30} height={8} r
          <rect x={room.x+15} y={room.y+room.h-55} width={room.w-30} height={12}
          <text x={room.x+room.w/2} y={room.y+room.h-45} textAnchor="middle" font
        </g>
      )}
    </g>
  );
```

```
}

// ── NOTIFICATION POPUP ──────────────────────────────────────────────
function OrderNotification({order,onClose,onAccept}){
  return(
    <div style={{
      position:"fixed",top:70,right:20,width:280,
      background:"#050d08",border:"2px solid #00ff88",borderRadius:6,
      boxShadow:"0 0 30px #00ff8833",zIndex:1000,fontFamily:"'Courier New',monosp
      animation:"slideIn 0.3s ease"
    }}>
      <style>{`@keyframes slideIn{from{transform:translateX(320px);opacity:0}to{t
      <div style={{background:"#00ff8822",padding:"10px 14px",borderBottom:"1px s
        <span style={{color:"#00ff88",fontSize:11,fontWeight:"bold",letterSpacing
        <button onClick={onClose} style={{background:"none",border:"none",color:"
      </div>
      <div style={{padding:14}}>
        <div style={{color:"#ffdd00",fontSize:10,marginBottom:8}}>Order #{order.i
        <div style={{color:"#aaa",fontSize:9,marginBottom:4}}>Customer: <span sty
        <div style={{color:"#aaa",fontSize:9,marginBottom:4}}>Item: <span style={
        <div style={{color:"#aaa",fontSize:9,marginBottom:10}}>Value: <span style
        <div style={{display:"flex",gap:8}}>
          <button onClick={onAccept} style={{flex:1,background:"#0a2a14",border:"
          <button onClick={onClose} style={{flex:1,background:"#1a0808",border:"1
        </div>
      </div>
    </div>
  );
}

// ── EDIT MODAL ──────────────────────────────────────────────────────
function EditModal({type,data,onSave,onClose}){
  const [form,setForm]=useState({...data});
  return(
    <div style={{position:"fixed",inset:0,background:"#000a",zIndex:2000,display:
      <div style={{background:"#080d08",border:"1px solid #00ff8844",borderRadius
        <div style={{color:"#00ff88",fontSize:11,letterSpacing:3,marginBottom:16}
        {Object.keys(form).filter(k=>k!=="id"&&k!=="emoji").map(k=>(
          <div key={k} style={{marginBottom:10}}>
            <div style={{fontSize:8,color:"#1a5a30",letterSpacing:2,marginBottom:
            <input value={form[k]} onChange={e=>setForm(f=>({...f,[k]:e.target.ty
              type={typeof form[k]==="number"?"number":"text"}
              style={{width:"100%",background:"#040a06",border:"1px solid #0a2a14
          </div>
        ))}
        <div style={{display:"flex",gap:8,marginTop:16}}>
          <button onClick={()=>onSave(form)} style={{flex:1,background:"#0a2a14",
```

```
          <button onClick={onClose} style={{flex:1,background:"transparent",borde
        </div>
      </div>
    </div>
  );
}

// ── MAIN ──────────────────────────────────────────────────────────────────────
export default function FactoryV3(){
  const [workers,setWorkers]=useState(()=>WORKER_DEFS.map(initWorker));
  const [boxes,setBoxes]=useState(()=>CONVEYORS.flatMap(c=>[mkBox(c.id),mkBox(c.i
  const [stockItems,setStockItems]=useState(INITIAL_STOCK);
  const [selectedId,setSelectedId]=useState(null);
  const [selectedRoom,setSelectedRoom]=useState(null);
  const [sideOpen,setSideOpen]=useState(true);
  const [logs,setLogs]=useState([
    {t:"00:00:00",msg:"🏭 Factory online. All systems go.",type:"ok"},
    {t:"00:00:01",msg:"👑 Dee Boss has assumed command.",type:"ok"},
    {t:"00:00:02",msg:"📦 Stock loaded: 2 Hoodies, 3 T-Shirts, 5 Caps.",type:"ok"
  ]);
  const [uptime,setUptime]=useState(0);
  const [ordersProcessing,setOrdersProcessing]=useState(3);
  const [ordersShipping,setOrdersShipping]=useState(1);
  const [ordersPending,setOrdersPending]=useState(5);
  const [income,setIncome]=useState(247.50);
  const [outgoing,setOutgoing]=useState(89.20);
  const [notification,setNotification]=useState(null);
  const [paused,setPaused]=useState(false);
  const [editModal,setEditModal]=useState(null); // {type,data,index}
  const [tick,setTick]=useState(0);
  const logRef=useRef(null);
  const tickRef=useRef(0);

  const CUSTOMERS=["Emma T.","James R.","Sophie K.","Oliver M.","Isla P.","Noah B

  const addLog=useCallback((msg,type="info")=>{
    setUptime(u=>{setLogs(p=>[...p.slice(-100),{t:fmt(u),msg,type}]);return u;});
  },[]);

  const triggerOrder=useCallback(()=>{
    const item=stockItems[rand(0,stockItems.length-1)];
    if(!item||item.qty<=0) return;
    const order={id:Math.floor(Math.random()*90000+10000),customer:randEl(CUSTOME
    setNotification(order);
  },[stockItems]);

  const acceptOrder=useCallback(()=>{
```

```
  if(!notification) return;
  setStockItems(prev=>prev.map(s=>s.id===notification.item.id?{...s,qty:Math.ma
  setOrdersPending(p=>p+1);
  setIncome(i=>+(i+notification.value).toFixed(2));
  addLog(`🛒 Order #${notification.id} accepted — ${notification.item.emoji} ${
  setNotification(null);
},[notification,addLog]);

// ── tick ──────────────────────────────────────────────────────────────────
useEffect(()=>{
  if(paused) return;
  const iv=setInterval(()=>{
    tickRef.current++;
    const t=tickRef.current;
    setTick(t);
    setUptime(u=>u+1);

    // boxes
    setBoxes(prev=>{
      let nb=prev.map(b=>({...b,t:b.t+0.016})).filter(b=>b.t<1.05);
      if(Math.random()<0.05){
        const c=CONVEYORS[rand(0,CONVEYORS.length-1)];
        nb=[...nb,mkBox(c.id)];
        if(c.to==="storage"){setOrdersShipping(s=>Math.max(0,s-1));setOrdersPro
        if(c.to==="packaging"){setOutgoing(o=>+(o+rand(5,20)).toFixed(2));}
      }
      return nb;
    });

    // workers
    setWorkers(prev=>prev.map(w=>{
      if(w.paused) return w;
      let nw={...w,walkPhase:w.walkPhase+1};
      if(nw.walking){
        const dx=nw.tx-nw.wx,dy=nw.ty-nw.wy,dist=Math.sqrt(dx*dx+dy*dy);
        if(dist<4){nw.walking=false;nw.wx=nw.tx;nw.wy=nw.ty;}
        else{const sp=nw.isBoss?2.2:1.6;nw.wx+=dx/dist*sp;nw.wy+=dy/dist*sp;}
      }
      if(!nw.walking&&rand(0,28)===0){
        const r=ROOMS[nw.roomId];
        nw.tx=r.x+rand(26,r.w-42);nw.ty=r.y+rand(26,r.h-34);nw.walking=true;
      }
      nw.progress+=rand(2,9);
      if(nw.progress>=100){
        nw.progress=0;nw.tasksToday++;
        nw.action=randEl(ROOM_ACTIONS[nw.roomId]);
        nw.animAction=randEl(ANIM_ACTIONS[nw.roomId]);
```

```
        nw.mood=randEl(["😊","😐","💪","🤔","😄"]);
        if(!nw.isBoss){const r=Math.random();nw.status=r<0.05?"idle":r<0.09?"bu
        if(nw.roomId==="orders"){setOrdersProcessing(p=>p+1);setOrdersPending(p
        if(nw.roomId==="packaging"){setOrdersShipping(s=>s+1);setOrdersProcessi
      }
      return nw;
    }));

    // random new order notification every ~25 ticks
    if(t%25===0&&Math.random()<0.6) triggerOrder();

    // log
    if(t%10===0){
      setWorkers(ws=>{
        const w=randEl(ws);
        setUptime(u=>{setLogs(p=>[...p.slice(-100),{t:fmt(u),msg:`${w.mood} ${w
        return ws;
      });
    }
  },500);
  return()=>clearInterval(iv);
},[paused,triggerOrder]);

useEffect(()=>{if(logRef.current)logRef.current.scrollTop=logRef.current.scroll

const selW=workers.find(w=>w.id===selectedId);
const sCol=s=>({active:"#00ff88",busy:"#ffaa00",idle:"#555",paused:"#ff4444"}[s

const sendToRoom=(wid,roomId)=>{
  const dest=ROOMS[roomId];
  setWorkers(prev=>prev.map(w=>w.id===wid?{...w,roomId,tx:dest.x+rand(28,dest.w
  const w=workers.find(x=>x.id===wid);
  addLog(`📡 CMD → ${w?.name}: Moved to ${dest.label}`,"ok");
};

const saveEdit=(form)=>{
  if(editModal.type==="worker"){
    setWorkers(prev=>prev.map(w=>w.id===editModal.data.id?{...w,...form}:w));
    addLog(`✏️ Worker "${form.name}" updated`,"ok");
  } else if(editModal.type==="stock"){
    setStockItems(prev=>prev.map((s,i)=>i===editModal.index?{...s,...form}:s));
    addLog(`✏️ Stock "${form.name}" updated`,"ok");
  }
  setEditModal(null);
};

const totalStock=stockItems.reduce((a,s)=>a+s.qty,0);
```

```
const stockValue=stockItems.reduce((a,s)=>a+s.qty*s.price,0);

// ── RENDER ──────────────────────────────────────────────────────────────
return(
  <div style={{display:"flex",flexDirection:"column",height:"100vh",background:

    {/* ── TOP MENU BAR ── */}
    <div style={{background:"#030805",borderBottom:"2px solid #0a2a14",padding:
      <div style={{marginRight:20,flexShrink:0}}>
        <div style={{fontSize:7,color:"#1a5a30",letterSpacing:4}}>AUTOBOT</div>
        <div style={{fontSize:13,fontWeight:"bold",color:"#00ff88",letterSpacin
      </div>
      <div style={{width:1,height:36,background:"#0a2a14",marginRight:16,flexSh
      {[
        {l:"STOCK ITEMS",  v:totalStock,           c:"#00aaff",  icon:"📦"},
        {l:"PROCESSING",   v:ordersProcessing,     c:"#ff6600",  icon:"⚙"},
        {l:"SHIPPING",     v:ordersShipping,       c:"#cc44ff",  icon:"🚚"},
        {l:"PENDING",      v:ordersPending,        c:"#ffdd00",  icon:"📋"},
        {l:"ACTIVE CREW",  v:`${workers.filter(w=>w.status==="active").length}/
        {l:"INCOME",       v:`£${income.toFixed(2)}`,  c:"#00ff88",  icon:"💰"}
        {l:"OUTGOING",     v:`£${outgoing.toFixed(2)}`, c:"#ff4444",  icon:"📪"
        {l:"BALANCE",      v:`£${(income-outgoing).toFixed(2)}`, c:income-outgo
      ].map(s=>(
        <div key={s.l} style={{display:"flex",alignItems:"center",gap:8,padding
          <span style={{fontSize:12}}>{s.icon}</span>
          <div>
            <div style={{fontSize:7,color:"#1a4a20",letterSpacing:1.5}}>{s.l}</
            <div style={{fontSize:12,fontWeight:"bold",color:s.c,letterSpacing:
          </div>
        </div>
      ))}
      <div style={{marginLeft:"auto",display:"flex",gap:8,flexShrink:0}}>
        <button onClick={()=>triggerOrder()} style={{background:"#0a2a14",borde
        <button onClick={()=>setPaused(p=>!p)} style={{background:paused?"#2a0a
      </div>
    </div>

    {/* ── BODY ── */}
    <div style={{display:"flex",flex:1,overflow:"hidden",position:"relative"}}>

      {/* ── FACTORY FLOOR ── */}
      <div style={{flex:1,overflow:"auto",background:"#05080a"}}>
        <svg width="800" height="490" style={{display:"block",minWidth:800,minH
          <defs>
            <pattern id="grid" width="28" height="28" patternUnits="userSpaceOn
              <path d="M28 0L0 0 0 28" fill="none" stroke="#080e08" strokeWidth
            </pattern>
```

```
    </defs>
    <rect width="800" height="490" fill="#060a07"/>
    <rect width="800" height="490" fill="url(#grid)"/>
    {CONVEYORS.map(c=><ConveyorBelt key={c.id} conv={c} boxes={boxes} tic
    {Object.values(ROOMS).map(room=>(
      <RoomBlock key={room.id} room={room} isSelected={selectedRoom===roo
        onClick={id=>setSelectedRoom(id===selectedRoom?null:id)} stockIte
    ))}
    {workers.map(w=>(
      <Character key={w.id} w={w} isSelected={selectedId===w.id}
        onClick={id=>setSelectedId(id===selectedId?null:id)}/>
    ))}
    {/* uptime strip */}
    <rect x={0} y={472} width={800} height={18} fill="#030805" opacity={0
    <text x={10} y={484} fontSize={7.5} fill="#1a4a20" fontFamily="'Couri
      ○ UPTIME {fmt(uptime)}  │  STOCK VALUE £{stockValue.toFixed(2)}  │
    </text>
  </svg>
</div>

{/* ── TOGGLE BUTTON ── */}
<button onClick={()=>setSideOpen(o=>!o)} style={{
  position:"absolute",right:sideOpen?268:0,top:"50%",transform:"translate
  background:"#050d08",border:"1px solid #0a2a14",borderRight:sideOpen?"n
  color:"#00ff88",width:18,height:60,cursor:"pointer",display:"flex",alig
  justifyContent:"center",fontSize:10,zIndex:10,borderRadius:sideOpen?"3p
  transition:"right 0.3s",flexShrink:0
}}>{sideOpen?"›":"‹"}</button>

{/* ── SIDE PANEL ── */}
<div style={{
  width:sideOpen?268:0,background:"#050d08",borderLeft:"1px solid #0a2a14
  display:"flex",flexDirection:"column",overflow:"hidden",flexShrink:0,
  transition:"width 0.3s",
}}>
  <div style={{width:268,display:"flex",flexDirection:"column",height:"10

    {/* Selected worker */}
    <div style={{borderBottom:"1px solid #0a2a14",padding:12,flexShrink:0
      <div style={{fontSize:8,color:"#00ff88",letterSpacing:3,marginBotto
      {selW?(
        <>
          <div style={{display:"flex",justifyContent:"space-between",alig
            <div>
              <div style={{fontSize:12,fontWeight:"bold",color:selW.color
              <div style={{fontSize:8,color:"#2a6a40"}}>{selW.role}</div>
            </div>
```

```
          <div style={{display:"flex",gap:4,alignItems:"center"}}>
            <div style={{fontSize:8,padding:"2px 6px",borderRadius:2,ba
            {!selW.isBoss&&<button onClick={()=>setEditModal({type:"wor
              style={{background:"transparent",border:"1px solid #1a3a2
            </div>
          </div>
          <div style={{background:"#0a1a0a",borderRadius:2,height:3,margi
            <div style={{width:`${selW.progress}%`,height:"100%",backgrou
          </div>
          <div style={{fontSize:8,color:"#1a4a2a",marginBottom:8}}>{selW.
          <div style={{display:"grid",gridTemplateColumns:"1fr 1fr 1fr",g
            {[["EFF",`${selW.efficiency}%`,selW.color],["TASKS",selW.task
              <div key={l} style={{background:"#040a06",padding:"4px",bor
                <div style={{fontSize:6,color:"#1a3a20"}}>{l}</div>
                <div style={{fontSize:10,color:c,fontWeight:"bold"}}>{v}<
              </div>
            ))}
          </div>
          {!selW.isBoss&&(
            <>
              <div style={{fontSize:7,color:"#1a5a30",letterSpacing:2,mar
              <div style={{display:"grid",gridTemplateColumns:"1fr 1fr 1f
                {Object.values(ROOMS).filter(r=>r.id!==selW.roomId&&r.id!
                  <button key={room.id} onClick={()=>sendToRoom(selW.id,r
                ))}
              </div>
              <button onClick={()=>setWorkers(prev=>prev.map(w=>w.id===se
                style={{width:"100%",background:"transparent",border:"1px
                {selW.paused?"▶ RESUME":"⏸ PAUSE WORKER"}
              </button>
            </>
          )}
          {selW.isBoss&&<div style={{fontSize:8,color:"#00ff88",textAlign
        </>
      ):(
        <div style={{fontSize:9,color:"#1a4a2a",textAlign:"center",paddin
      )}
    </div>

    {/* Stock items */}
    <div style={{borderBottom:"1px solid #0a2a14",padding:12,flexShrink:0
      <div style={{display:"flex",justifyContent:"space-between",alignIte
        <div style={{fontSize:8,color:"#ffdd00",letterSpacing:2}}>📦 STOC
        <button onClick={()=>setStockItems(prev=>[...prev,{id:`s${Date.no
          style={{background:"transparent",border:"1px solid #ffdd0044",c
      </div>
      {stockItems.map((s,i)=>(
```

```
          <div key={s.id} style={{display:"flex",justifyContent:"space-betw
            <span style={{fontSize:9,color:"#ffdd88"}}>{s.emoji} {s.name}</
            <div style={{display:"flex",alignItems:"center",gap:6}}>
              <span style={{fontSize:9,color:"#ffdd00",fontWeight:"bold"}}>
              <span style={{fontSize:8,color:"#2a5a20"}}>£{s.price}</span>
              <button onClick={()=>setEditModal({type:"stock",data:{name:s.
                style={{background:"none",border:"none",color:"#2a5a30",cur
            </div>
          </div>
        ))}
      </div>

      {/* Worker roster */}
      <div style={{flex:1,overflow:"auto"}}>
        <div style={{fontSize:8,color:"#1a5a30",letterSpacing:3,padding:"8p
        {workers.map(w=>(
          <div key={w.id} onClick={()=>setSelectedId(w.id===selectedId?null
            style={{display:"flex",alignItems:"center",gap:7,padding:"5px 1
              background:selectedId===w.id?"#0a1f0e":"transparent",
              borderLeft:selectedId===w.id?`2px solid ${w.color}`:"2px soli
            <div style={{width:7,height:7,borderRadius:"50%",background:sCo
            <div style={{flex:1,minWidth:0}}>
              <div style={{fontSize:9,color:w.color,fontWeight:"bold",white
              <div style={{fontSize:7,color:"#2a5a35",whiteSpace:"nowrap",o
            </div>
            <div style={{fontSize:7,color:sCol(w.status),flexShrink:0}}>{w.
          </div>
        ))}
      </div>

      {/* Activity log */}
      <div style={{height:110,borderTop:"1px solid #0a2a14",padding:"8px 12
        <div style={{fontSize:7,color:"#1a5a30",letterSpacing:3,marginBotto
        <div ref={logRef} style={{flex:1,overflowY:"auto",scrollbarWidth:"n
          {logs.map((l,i)=>(
            <div key={i} style={{display:"flex",gap:5,marginBottom:2}}>
              <span style={{fontSize:7,color:"#1a3a20",flexShrink:0}}>{l.t}
              <span style={{fontSize:8,color:{ok:"#00ff88",warn:"#ffaa00",e
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
</div>

{/* ── NOTIFICATION ── */}
```

```
      {notification&&(
        <OrderNotification order={notification} onClose={()=>setNotification(null
      )}

      {/* ── EDIT MODAL ── */}
      {editModal&&(
        <EditModal type={editModal.type} data={editModal.data} onSave={saveEdit}
      )}
    </div>
  );
}
```